

SALVANDO A ELEIÇÃO NORTE-AMERICANA DE 1936 COM DATA SCIENCE

Guia Técnico da Fase 2

Modelagem Preditiva

Como construir a correção da previsão de 1936 usando o resultado eleitoral de 1932 como âncora, com Regressão Linear em Scikit-Learn — passo a passo, com pontos de atenção.

 Fase 2 — Membro responsável pela modelagem

 Prazo interno: 15/09 a 21/09

 Python · Pandas · NumPy · Scikit-Learn

1 O que a Fase 2 precisa entregar

De acordo com o regulamento oficial do hackathon e o Plano de Ação da equipe, este é o escopo exato da sua etapa.

O QUE FAZER

Construir um modelo de **Regressão Linear simples** com **Scikit-Learn**, usando o resultado eleitoral de **1932** como âncora para corrigir a previsão da pesquisa Literary Digest de **1936**, e calcular as métricas de erro dessa correção.

Item	Descrição
Depende de	Notebook da Fase 1 (Membro A): dataset limpo/exportado, proporções por estado e diagnóstico de viés já prontos.
Entregável	Script/notebook com o modelo implementado, métricas de erro (MAE/RMSE) e tabela de resultados por estado, exportável.
Roda em paralelo com	Membro C, que escreve Introdução/Metodologia do resumo ao mesmo tempo — mas a seção de Resultados dele só fecha depois que você entregar os números finais.
Prazo	15/09 (ter) a 21/09 (seg) — 7 dias.

Critérios de aceitação

- Regressão Linear implementada em Scikit-Learn usando 1932 como âncora.
- Métricas de erro calculadas (MAE, RMSE) comparando previsão Literary Digest × previsão corrigida × resultado real de 1936.
- Script/notebook reproduzível — roda do início ao fim sem erro ("Restart & Run All").
- Resultado por estado disponível em tabela exportável.
- Conclusão objetiva: a correção reduziu o erro? Em quanto?

COMBINE COM A EQUIPE

Avise o **Membro C** assim que as métricas finais estiverem prontas — mesmo antes do prazo — e combine com o **Membro D** como repassar os gráficos e números para o pôster. Exportar os gráficos já como imagem (PNG) durante a Fase 2 economiza o trabalho dele depois.

2 Antes de começar

O que a Fase 1 já deixou pronto para você usar, e o que ainda precisa ser feito por você — nenhuma das duas colunas pode ser pulada.

✓ Já pronto (Fase 1)

- `poll_1936_clean.csv` — pesquisa Digest 1936 limpa
- `actual_1936_clean.csv` — resultado oficial de 1936
- `analysis_1936.csv` — os dois cruzados, com `landon_share_two_party`, `actual_landon_share` e `landon_error` já calculados
- `bias_diagnostics_1936.csv` — TVD de composição e não-resposta por estado

⚠ Ainda falta (é seu trabalho)

- Extrair e limpar a aba "**1932**" do Excel bruto (pesquisa + resultado oficial daquele ano) — ninguém fez isso ainda
- Calcular as proporções two-party de 1932
- Treinar a regressão e aplicá-la em 1936
- Calcular MAE/RMSE e montar a tabela final por estado

🚫 ARMADILHA Nº 1 — NÃO CONFUNDA AS DUAS FONTES DE "1932"

O arquivo `poll_1936_clean.csv` já tem colunas como `landon_1932_rep`, `roosevelt_1932_dem` etc. — isso é o **voto retrospectivo** que os respondentes da pesquisa de 1936 disseram ter dado em 1932 (autodeclarado, dentro da mesma pesquisa). Isso **não** é a pesquisa real de 1932 nem o resultado real de 1932. A âncora que o regulamento pede é a aba "**1932**" do Excel bruto, com sua própria pesquisa (Hoover × Roosevelt) e seu próprio resultado oficial — que ainda não foi extraída por ninguém da equipe.

3 Passo a passo técnico

Nove etapas, na ordem em que devem ser feitas. Siga o mesmo estilo do notebook da Fase 1: uma célula de markdown explicando a decisão antes de cada bloco de código, e `assert` s para validar cada etapa antes de seguir para a próxima.

1 Preparar o ambiente e carregar o que já existe

Comece um novo notebook, `notebooks/02_fase2_modelagem.ipynb`, e carregue os artefatos da Fase 1 sem alterá-los.

python

```
# --- bibliotecas ---
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error

# --- dados já prontos da Fase 1 ---
df_1936 = pd.read_csv("../data/processed/analysis_1936.csv")
RAW_PATH = "../data/raw/LitDigestFull.xlsx"
xls = pd.ExcelFile(RAW_PATH)

df_1936.shape, df_1936.columns.tolist()[5:]
```

DICA

`analysis_1936.csv` já contém tudo que você precisa do lado de 1936: `landon_share_two_party` (previsão da pesquisa) e `actual_landon_share` (resultado real). Você só vai *adicionar* uma coluna de previsão corrigida a essa base — não precisa recalcular nada da Fase 1.

2 Extrair a aba "1932" — bloco da pesquisa Digest

A aba "1932" do Excel bruto tem dois blocos, como a aba 1936 tinha. Mapeamento de colunas confirmado diretamente no arquivo:

Coluna	Conteúdo
A	Estado — atenção: vem em CAIXA ALTA ("ALABAMA"), diferente da aba 1936
B	Electoral Votes
C	Hoover — contagem de respostas da pesquisa Digest
D–G	H_Rep, H_Dem, H_Other, H_No vote — voto retrospectivo de 1928 dos eleitores de Hoover (4 categorias)
J	Roosevelt — contagem de respostas da pesquisa Digest
K–N	R_Rep, R_Dem, R_Other, R_No vote
Q–V	Candidatos menores: Thomas, Reynolds, Coxey, Upshaw, Foster, Misc
W	Total_Other (soma dos menores)
Y	Total — total de respondentes da pesquisa em 1932

python

```
# --- linha 0 e 1 sao cabecalho em duas linhas, igual a aba 1936 ---
raw_1932 = pd.read_excel(xls, sheet_name="1932", header=None)

poll_1932_cols = {
    "state": 0, "electoral_vote": 1,
    "hoover": 2, "roosevelt": 9,
    "grand_total": 24,
}

df_poll_1932 = raw_1932.iloc[2:, list(poll_1932_cols.values())].copy()
df_poll_1932.columns = list(poll_1932_cols.keys())

# estado em title case, para bater com o padrao ja usado em 1936
df_poll_1932["state"] = df_poll_1932["state"].str.title()
```

⚠ ATENÇÃO AO "CASO" DO TEXTO

Dentro da própria aba 1932, o bloco da pesquisa está em CAIXA ALTA e o bloco oficial (Wikipedia) já está em title case. Se você não normalizar os dois para o mesmo formato antes do merge, estados "somem" silenciosamente — o `merge` não vai dar erro, só vai descartar as linhas que não combinaram exatamente.

3 Extrair a aba "1932" — bloco oficial (Wikipedia)

O bloco oficial começa na coluna **AB** (índice 27). Repare que a ordem dos candidatos aqui é **invertida** em relação ao bloco da pesquisa: primeiro vem Roosevelt, depois Hoover.

Coluna	Conteúdo
AB	Estado (title case)
AC	Electoral votes
AD	Roosevelt — votos oficiais
AE	%
AF–AG	Electoral votes / Hoover — votos oficiais
AH	%

python

```
official_1932_cols = {
    "state": 27, "roosevelt_votes": 29,
    "hoover_votes": 32, "total_votes": 43,
}
df_actual_1932 = raw_1932.iloc[2:, list(official_1932_cols.values())].copy()
df_actual_1932.columns = list(official_1932_cols.keys())
df_actual_1932 = df_actual_1932.dropna(subset=["state"])
```

4 Limpeza e validação — replicando o rigor da Fase 1

Antes de unir os dois blocos, trate as mesmas três pendências que a Fase 1 tratou na aba 1936:

- **Linhas especiais:** "State Unknown" e "Totals:" aparecem no fim da aba 1932 também — separe-as, não descarte, e não deixe entrar na análise por estado.
- **District of Columbia:** a aba 1932 inclui "District of Columbia", mas a análise de 1936 (e toda a Fase 1) usa só os **48 estados**, sem DC. Para comparar as mesmas unidades nos dois lados do modelo, exclua DC também de 1932.
- **Traços ("-" ou "-"):** confira se aparecem nas colunas de votos/electoral votes dos candidatos menores no bloco oficial, do mesmo jeito que a Fase 1 fez — audite antes de converter para número.

python

```
# linhas especiais fora da analise por estado
special_states = ["State Unknown", "Totals:", "District Of Columbia"]

df_poll_1932_clean = df_poll_1932[~df_poll_1932["state"].isin(
    [s.upper() for s in special_states] + special_states
)].reset_index(drop=True)

df_actual_1932_clean =
df_actual_1932[~df_actual_1932["state"].isin(special_states)].reset_index(drop=True)

# tipos numericos + tratamento de "-"
for col in ["hoover", "roosevelt", "grand_total"]:
    df_poll_1932_clean[col] = pd.to_numeric(df_poll_1932_clean[col], errors="coerce")

for col in ["roosevelt_votes", "hoover_votes", "total_votes"]:
    df_actual_1932_clean[col] = (
        df_actual_1932_clean[col].astype(str).str.replace("-", "0").str.replace("-", "0")
    )
    df_actual_1932_clean[col] = pd.to_numeric(df_actual_1932_clean[col], errors="coerce")
```

```
# mesma logica de assert da Fase 1: 48 estados, sem duplicata, mesmos nomes
assert len(df_poll_1932_clean) == 48, f"esperado 48 estados na pesquisa 1932, encontrado {len(df_poll_1932_clean)}"
assert len(df_actual_1932_clean) == 48, f"esperado 48 estados no oficial 1932, encontrado {len(df_actual_1932_clean)}"
assert df_poll_1932_clean["state"].is_unique
assert set(df_poll_1932_clean["state"]) == set(df_actual_1932_clean["state"]), "nomes de estado divergem"
assert set(df_poll_1932_clean["state"]) == set(df_1936["state"]), "universo de estados diferente do usado em 1936"
print("Estrutura da aba 1932 validada: OK")
```

⚠ ARMADILHA Nº 2 — NÃO PULE OS ASSERTS

Se o merge por nome de estado falhar silenciosamente (por causa de caixa alta/baixa ou de "District of Columbia" sobrando), o modelo treina com **menos de 48 pontos** sem avisar — e o coeficiente da regressão sai errado sem nenhum erro no console. O último `assert` acima (comparando com o universo de estados de `df_1936`) é o que garante que os dois lados do seu modelo falam da mesma coisa.

5 Unir os blocos e calcular as proporções (two-party)

Use exatamente a mesma metodologia que a Fase 1 já usou em 1936 — participação entre os **dois principais candidatos** — para que os dois anos fiquem comparáveis no mesmo eixo.

python

```
df_1932 = df_poll_1932_clean.merge(df_actual_1932_clean, on="state", how="inner")
assert len(df_1932) == 48

# previsao da pesquisa (o que a Digest "achava" que ia acontecer)
df_1932["hoover_share_two_party_1932"] = (
    df_1932["hoover"] / (df_1932["hoover"] + df_1932["roosevelt"])
)

# resultado real (o que de fato aconteceu)
df_1932["actual_hoover_share_1932"] = (
    df_1932["hoover_votes"] / (df_1932["hoover_votes"] + df_1932["roosevelt_votes"])
)
```

💡 POR QUE "TWO-PARTY"?

Republicano (Hoover/Landon) é o candidato que estamos tentando prever melhor, e o Data Dictionary do gabarito usa exatamente essa mesma convenção (participação entre os dois principais partidos) nas colunas de ajuste. Manter o mesmo critério nos dois anos é o que permite aplicar o coeficiente aprendido em 1932 diretamente sobre a coluna `landon_share_two_party` que já existe em `analysis_1936.csv`.

6 Treinar a Regressão Linear com 1932 como âncora

Este é o coração da Fase 2. A ideia: a pesquisa Digest tinha um viés sistemático (superestimava o Republicano) — e esse viés já aparecia em 1932. Um modelo simples de calibração aprende, a partir de 1932, a relação entre "o que a

pesquisa previu" e "o que realmente aconteceu", e depois aplicamos essa mesma relação em 1936.

python

```
X_1932 = df_1932[["hoover_share_two_party_1932"]] # preditor (1 variavel = regressao "simples")
y_1932 = df_1932["actual_hoover_share_1932"]      # alvo

model = LinearRegression()
model.fit(X_1932, y_1932)

print(f"Intercepto (a): {model.intercept_:.4f}")
print(f"Coeficiente (b): {model.coef_[0]:.4f}")
# previsao_corrigida = a + b * previsao_da_pesquisa
```

🚩 O QUE ESSE MODELO REPRESENTA

É exatamente o formato $y = a + b \cdot x$ descrito no Data Dictionary do gabarito (lá aparece como exemplo o ajuste " $y = 1,4 + 0,88 \cdot \text{previsão Digest}$ "). Seu `intercept_` e `coef_` não precisam bater com esse valor exato — eles vêm do seu próprio tratamento de dados — mas a lógica de calibração é a mesma: um único preditor (a previsão bruta da pesquisa), por isso é chamada de regressão **simples**.

💡 POR QUE NÃO PRECISA DE CROSS-VALIDATION

Você já tem um split treino/teste legítimo "de graça": o modelo **treina** nos 48 pares de 1932 e é **testado** nos 48 pares de 1936 — anos diferentes, fora da amostra de treino. Não é necessário (nem esperado, para uma regressão "simples" de um hackathon introdutório) fazer k-fold ou separar 1932 em treino/validação.

7 Aplicar o modelo em 1936 e gerar a previsão corrigida

python

```
X_1936 = df_1936[["london_share_two_party"]] # ja existe em analysis_1936.csv

df_1936["london_share_corrected"] = model.predict(X_1936)

df_1936[["state", "london_share_two_party", "london_share_corrected", "actual_london_share"]].head()
```

⚠️ CONFIRA A ESCALA ANTES DE APLICAR

Se você calculou as proporções de 1932 em fração (0 a 1) e as de 1936 também em fração (é o padrão de `analysis_1936.csv`), tudo bem. Mas se em algum ponto você multiplicou por 100 (percentual), o modelo vai prever fora da escala esperada. Confira com um `.describe()` nas duas colunas de entrada antes de treinar — os dois devem estar na mesma unidade.

8 Métricas de erro: bruto × corrigido × real

Compare as três colunas lado a lado, estado por estado, e resuma com MAE e RMSE — os mesmos indicadores pedidos no critério de aceitação.

python

```
mae_raw = mean_absolute_error(df_1936["actual_london_share"], df_1936["london_share_two_party"])
mae_corr = mean_absolute_error(df_1936["actual_london_share"], df_1936["london_share_corrected"])

rmse_raw = mean_squared_error(df_1936["actual_london_share"], df_1936["london_share_two_party"],
squared=False)
rmse_corr = mean_squared_error(df_1936["actual_london_share"], df_1936["london_share_corrected"],
squared=False)

reducao_mae = (mae_raw - mae_corr) / mae_raw * 100

print(f"MAE bruto:      {mae_raw:.4f}")
print(f"MAE corrigido:  {mae_corr:.4f}")
print(f"Reducao do erro: {reducao_mae:.1f}%")
print(f"RMSE bruto:      {rmse_raw:.4f} | RMSE corrigido: {rmse_corr:.4f}")
```

Complemente com uma leitura estado a estado — quantos estados tiveram o **vencedor previsto corretamente** antes e depois da correção. Esse número concreto ("a correção acertou o vencedor em X estados a mais") é ótimo material para o resumo acadêmico do Membro C e para o pôster do Membro D.

python

```
df_1936["acerto_bruto"] = (df_1936["london_share_two_party"] > 0.5) ==
(df_1936["actual_london_share"] > 0.5)
df_1936["acerto_corrigido"] = (df_1936["london_share_corrected"] > 0.5) ==
(df_1936["actual_london_share"] > 0.5)

print("Estados com vencedor correto (bruto):", df_1936["acerto_bruto"].sum(), "/ 48")
print("Estados com vencedor correto (corrigido):", df_1936["acerto_corrigido"].sum(), "/ 48")
```

9 Exportar tabela final e gráficos

Feche a etapa exportando os artefatos que os Membros C e D vão reaproveitar — nomeação sugerida, seguindo o padrão de pastas já usado na Fase 1.

python

```
cols_export = ["state", "london_share_two_party", "london_share_corrected",
               "actual_london_share", "acerto_bruto", "acerto_corrigido"]
df_1936[cols_export].to_csv("../data/processed/regression_results_1936.csv", index=False)

# grafico comparativo por estado, para o poster do Membro D
fig, ax = plt.subplots(figsize=(7, 11))
ordered = df_1936.sort_values("actual_london_share")
ax.scatter(ordered["london_share_two_party"], ordered["state"], label="Previsão bruta",
color="tab:red")
ax.scatter(ordered["london_share_corrected"], ordered["state"], label="Previsão corrigida",
color="tab:blue")
ax.scatter(ordered["actual_london_share"], ordered["state"], label="Resultado real", color="black",
marker="|")
ax.axvline(0.5, color="gray", linestyle="--", linewidth=1)
ax.legend()
plt.tight_layout()
plt.savefig("../figures/correcao_por_estado.png", dpi=150)
```



FECHE COM UMA CONCLUSÃO OBJETIVA, EM MARKDOWN, NO FIM DO NOTEBOOK

Uma frase com número já resolve o critério de aceitação: "a correção via regressão linear reduziu o MAE de X para Y (redução de Z%), e acertou o vencedor em N estados a mais do que a previsão bruta da pesquisa."

4 Pontos de atenção — resumo

As armadilhas mais prováveis nesta fase, e como evitá-las.

⚠ CONFUNDIR AS DUAS FONTES DE "1932"

O voto retrospectivo já embutido em `poll_1936_clean.csv` (colunas `*_1932_rep`, `*_1932_dem` etc.) não é a pesquisa real de 1932. A âncora exigida vem da aba "1932" do Excel bruto, que você mesmo precisa extrair e limpar (Passos 2 a 5).

⚠ MERGE SILENCIOSO PERDENDO ESTADOS

Caixa alta em um lado ("ALABAMA") e title case no outro ("Alabama") faz o `merge` descartar linhas sem lançar erro. Sempre confira o tamanho do resultado com `assert len(df) == 48` logo após cada junção.

⚠ INCLUIR DISTRICT OF COLUMBIA POR ENGANO

A aba 1932 tem DC, a análise de 1936 não. Exclua DC do treino de 1932 para manter os dois lados do modelo no mesmo universo de 48 estados.

⚠ ESCALA INCONSISTENTE ENTRE OS ANOS

Proporções de 1932 e 1936 precisam estar na mesma unidade (fração 0–1) antes de treinar e aplicar o modelo — confira com `.describe()` nas duas colunas de entrada.

⚠ TRAÇOS ("-" / "-") NAS COLUNAS NUMÉRICAS

O bloco oficial da aba 1932 pode ter traços em colunas de candidatos menores, como aconteceu na aba 1936. Audite antes de converter para número — não assuma que "-" sempre significa zero.

5 Checklist final de entrega

Confira cada item antes de avisar o Membro C que os números estão prontos.

- ☐ Aba "1932" (pesquisa + oficial) extraída, limpa e validada com asserts — 48 estados, sem DC, sem duplicata.
- ☐ Proporções two-party de 1932 e 1936 calculadas com a mesma metodologia.
- ☐ Regressão Linear (Scikit-Learn) treinada com 1932 como âncora — um único preditor.
- ☐ Modelo aplicado sobre a previsão bruta de 1936, gerando a previsão corrigida por estado.
- ☐ MAE e RMSE calculados para bruto × real e corrigido × real, com a redução percentual do erro.
- ☐ Contagem de estados com vencedor previsto corretamente, antes e depois da correção.

☐ Tabela por estado exportada em CSV (`data/processed/`) e gráfico(s) exportado(s) em PNG (`figures/`).

☐ Notebook roda do início ao fim sem erro ("Restart & Run All"), com células de markdown explicando cada decisão.

☐ Conclusão objetiva escrita em markdown: quanto a correção reduziu o erro.

☐ Membro C avisado com os números finais; Membro D combinado sobre gráficos/números para o pôster.

Prazo: 21/09

Entrega final da equipe: 27/09